



Degree Project in Computer Science and Engineering

Second cycle, 30 credits

Adaptive Hybrid RAG

Optimizing the Cost-Performance Trade-off via Learned Query
Routing

DANTE WESSLUND

Adaptive Hybrid RAG

Optimizing the Cost-Performance Trade-off via Learned Query Routing

DANTE WESSLUND

Master's Programme, Machine Learning, 120 credits

Date: May 24, 2026

Supervisors: Arvid Eriksson, Peng Zhang

Examiner: Pawel Herman

School of Electrical Engineering and Computer Science

Host company: Ericsson

Swedish title: Adaptiv hybrid-RAG

Swedish subtitle: Optimering av kostnads-prestanda-avvägningen med inlärd
frågerouting

Abstract

Retrieval-Augmented Generation (RAG) is a common approach for grounding large language models in external knowledge. Production systems increasingly combine cheap vector retrieval with more expensive graph-enhanced retrieval, because graph-enhanced retrieval can help answer questions whose evidence is split across documents but uses substantially more compute than vector retrieval. Applying graph-enhanced retrieval to every query wastes resources on questions that the cheap path could already handle, while always using vector retrieval risks missing questions that benefit from the richer retrieval mode.

This thesis studies whether the choice between a cheap vector retrieval mode and a more expensive graph-enhanced retrieval mode can be predicted from the query text alone. The setting is the 2WikiMultihopQA benchmark inside the LightRAG retrieval framework, with LightRAG’s Naive mode as the cheap path and its Mix mode as the graph-enhanced path. Routing labels are constructed by running both retrieval modes on every benchmark question and using a large language model as a judge to compare each mode’s answer against the gold answer. The resulting binary routing dataset is split jointly by question type and routing label, and is used to train and evaluate two query-only routers: a TF-IDF logistic-regression baseline and a fine-tuned ModernBERT classifier.

On a held-out test split of 193 queries, the ModernBERT router achieves higher point estimates than the TF-IDF baseline on aggregate routing-label classification metrics, including area under the precision-recall curve (0.769 vs. 0.742), balanced routing-label accuracy (0.798 vs. 0.784), and F1 on the minority class (0.699 vs. 0.650), with differences that are stable across three training seeds. Treated as offline routing policies and evaluated along two routed-cost dimensions, both learned routers route at substantially lower mean cost than always using the Mix mode. The ModernBERT router reduces mean routed execution time by 39 % and estimated mean routed model-token usage by 60 % relative to the always-Mix baseline. Mean token usage is estimated from per-mode means measured on an instrumented subset of $N = 150$ queries spanning all four 2WikiMultihopQA question types. These metrics evaluate whether the router selects the judge-derived minimum sufficient retrieval mode, not an independently re-judged final answer accuracy of the routed system. Within this experimental setting, the retrieval-mode distinction is partly predictable from the query text alone, and a lightweight encoder classifier is a promising routing layer for adaptive Hybrid RAG.

Keywords

Retrieval-Augmented Generation, Hybrid RAG, Query routing, Modern-BERT, 2WikiMultihopQA, LLM-as-a-Judge, Routing-label classification, Routed cost

Sammanfattning

Retrieval-Augmented Generation (RAG) är en vanlig arkitektur för att förankra stora språkmodeller i externt material. I moderna system kombineras allt oftare billig vektorbaserad sökning med dyrare grafbaserad hämtning, eftersom grafbaserad hämtning kan hjälpa till att besvara frågor vars belägg är fördelade över flera dokument men kräver väsentligt mer beräkningsresurser än vektorbaserad sökning. Att tillämpa den grafbaserade hämtningen på varje fråga slösar resurser på frågor som den billigare metoden redan klarar, medan att alltid använda enbart vektorsökning riskerar att missa frågor som drar nytta av det mer omfattande hämtningsläget.

Denna avhandling undersöker om valet mellan ett billigt vektorbaserat hämtningsläge och ett dyrare grafbaserat hämtningsläge kan förutsägas enbart utifrån frågetexten. Experimenten utförs på riktmärket 2WikiMultihopQA i hämtningsramverket LightRAG, där LightRAG:s *Naive*-läge utgör den billiga vägen och dess *Mix*-läge den grafbaserade vägen. Routingetiketter konstrueras genom att köra båda hämtningslägena på varje fråga och låta en stor språkmodell som domare jämföra respektive svar mot facit. Det resulterande binära routingdatasetet delas upp gemensamt efter frågetyp och routingetikett och används för att träna och utvärdera två routrar som endast ser frågetexten, nämligen en TF-IDF-baserad logistisk regressionsmodell som baslinje och en finjusterad ModernBERT-klassificerare.

På en testmängd om 193 frågor uppnår ModernBERT-routern högre punkttestimat än TF-IDF-baslinjen på aggregerade routingklassificeringsmått, däribland arean under precision-recall-kurvan (0,769 mot 0,742), balanserad routingnoggrannhet (0,798 mot 0,784) och F1 på minoritetsklassen (0,699 mot 0,650), med skillnader som är stabila över tre träningsfrön. Som offline-routingpolicyer utvärderade längs två kostnadsdimensioner ger båda routrarna betydligt lägre medelkostnad än att alltid använda Mix-läget. ModernBERT-routern minskar medellexekveringstiden med 39 % och den uppskattade användningen av modelltoken med 60 % relativt always-Mix-baslinjen. Tokenanvändningen uppskattas från medelvärden per läge uppmätta på en instrumenterad delmängd om $N = 150$ frågor som täcker samtliga fyra frågetyper i 2WikiMultihopQA. Dessa mått utvärderar om routern väljer den bedömda minsta tillräckliga hämtningsmetoden, inte en separat ombedömd slutlig svarskorrekthet för hela routingsystemet. I denna experimentella miljö är valet av hämtningsläge delvis förutsägbart enbart utifrån frågetexten, och en lättviktig encoder-klassificerare är ett lovande routinglager för adaptiv hybrid-RAG.

Nyckelord

Retrieval-Augmented Generation, hybrid-RAG, frågeroutning, ModernBERT, 2WikiMultihopQA, LLM-som-domare, routingklassificering, routad kostnad

Acknowledgments

I would like to thank my KTH supervisor Arvid Eriksson and my Ericsson supervisor Peng Zhang for their guidance, feedback, and continuous support throughout this thesis project.

Stockholm, May 2026
Dante Wesslund

Contents

1	Introduction	1
1.1	Background	1
1.2	Research problem	2
1.2.1	Problem definition	2
1.2.2	Research question	3
1.3	Purpose	3
1.4	Goals	4
1.5	Research methodology	4
1.6	Delimitations	5
1.7	Structure of the thesis	6
2	Background	7
2.1	Retrieval-augmented generation	7
2.2	Graph-enhanced retrieval	8
2.2.1	Graph-based RAG systems	9
2.2.2	LightRAG <i>Naive</i> and <i>Mix</i> modes	9
2.3	Adaptive retrieval and query routing	10
2.4	Language models and text classifiers	11
2.4.1	Encoder-only transformers and ModernBERT	12
2.4.2	TF-IDF logistic regression	12
2.5	Related work	13
3	Methods	15
3.1	Data	15
3.1.1	2WikiMultihopQA	15
3.1.2	Preprocessing and indexing	16
3.2	Routing-label pipeline	16
3.2.1	Dual-mode retrieval	17
3.2.2	LLM-as-a-Judge labeling	18

3.3	Binary routing dataset	18
3.4	Router models	19
3.4.1	TF-IDF logistic-regression baseline	20
3.4.2	ModernBERT router	20
3.5	Evaluation	21
3.5.1	Held-out classification metrics	22
3.5.2	Offline routing policy	23
4	Implementation	25
4.1	Model training	25
4.1.1	TF-IDF logistic-regression baseline	25
4.1.2	ModernBERT router	26
4.2	Held-out classification experiment	26
4.3	Offline routed-cost experiment	27
4.4	Results	28
4.5	Discussion	37
5	Conclusions and future work	43
5.1	Conclusions	43
5.2	Limitations and future work	45
5.2.1	Token-cost measurement scope	45
5.2.2	Judge labels	45
5.2.3	Final answer accuracy of routed policies	45
5.2.4	Generalizability	46
5.2.5	Routing beyond the query	46
5.2.6	Calibration and abstention	47
5.3	Reflections	47
	References	49
A	Source code	53
B	Experimental configuration	55
B.1	Judge prompt	55

List of Figures

2.1	Stylized illustration of vector retrieval (<i>Naive</i>) versus graph-enhanced retrieval (<i>Mix</i>) on a two-hop question. Vector retrieval matches chunks individually against the query, while the graph-enhanced mode can follow entity-relation edges that connect facts living in different documents.	10
2.2	Query routing between LightRAG’s cheap <i>Naive</i> path and its more expensive <i>Mix</i> path. The router observes only the query and selects a retrieval mode before retrieval runs. The labels are operational. <i>Mix</i> beneficial means that <i>Mix</i> was judged beneficial relative to <i>Naive</i> under the experimental pipeline. . .	11
3.1	End-to-end pipeline from 2WikiMultihopQA samples to router evaluation. Each step writes its output to disk so the remaining steps can be re-run without repeating expensive earlier steps.	17
4.1	Precision–recall curves on the held-out test split for the term frequency-inverse document frequency (TF-IDF) baseline and the three ModernBERT training seeds. The dashed horizontal line is the test-set base rate of <code>mix_required</code>	32
4.2	Receiver-operating-characteristic curves on the held-out test split.	33
4.3	F1 on <code>mix_required</code> as a function of the decision threshold on the held-out test split. Dots mark the validation-selected threshold for each model: TF-IDF $\tau^* = 0.48$, ModernBERT seed 42 $\tau^* = 0.24$, seed 7 $\tau^* = 0.36$, seed 13 $\tau^* = 0.42$	33
4.4	Mean routed execution time on the held-out test split as the decision threshold sweeps. The two horizontal dashed lines are the always- <i>Naive</i> and always- <i>Mix</i> reference levels.	34

4.5	Routing-label F1 on the <code>mix_required</code> class versus mean routed execution time on the held-out test split. Each curve traces the operating points obtained by sweeping the decision threshold. Stars mark the validation-selected thresholds. Static <i>always-Naive</i> and <i>always-Mix</i> policies are shown as reference points.	35
4.6	Area under the precision-recall curve (AUPRC) on the <code>mix_required</code> class, broken down by 2WikiMultihopQA question type on the held-out test split. ModernBERT bars show the mean over three training seeds with error bars at one standard deviation. The <code>comparison</code> and <code>inference</code> slices have too few <code>mix_required</code> test rows to give an informative AUPRC and are omitted.	37

List of Tables

3.1	Label-flow summary from raw benchmark samples to the final binary routing dataset. The binary router is trained only on <code>naive_enough</code> and <code>mix_required</code> rows. The <code>mix_required</code> label is operational and means <i>Mix-beneficial</i> under the judged LightRAG setup.	19
3.2	Routing dataset summary by split and 2WikiMultihopQA question type. Each cell shows the total number of judged binary examples followed by the count of those examples that are labeled <code>mix_required</code>	20
4.1	Routing-label classification performance on the held-out test split. P_{mix} , R_{mix} , and $F1_{\text{mix}}$ denote precision, recall, and F1 on the <code>mix_required</code> class. Accuracy values are routing-label accuracies, not final answer accuracies. For the constant-score static baselines, area under the receiver operating characteristic curve (AUROC) equals 0.500 and AUPRC equals the test-set <code>mix_required</code> base rate of 0.254. The type-only diagnostic baseline is non-deployable because it uses 2WikiMultihopQA metadata rather than only query text. For TF-IDF, bracketed values are 95 % bootstrap intervals over 1 000 test-set resamples. For ModernBERT, values are reported as mean $\pm$ standard deviation across three training seeds.	30

4.2	Offline routed-cost and policy diagnostics on the held-out test split. Mean tokens are estimated from per-mode token means measured on the instrumented $N = 150$ query subset (Section 3.2.1). The token saving column reports the reduction in mean tokens relative to the always- <i>Mix</i> baseline. Under-routing means routing a <code>mix_required</code> query to <i>Naive</i> . Over-routing means routing a <code>naive_enough</code> query to <i>Mix</i> . Under-routing is the error type most directly associated with answer-quality risk, while over-routing is primarily a cost error. ModernBERT values are the mean over three training seeds $\pm$ standard deviation.	31
4.3	Per-question-type breakdown on the test split. P_{mix} , R_{mix} , and $F1_{\text{mix}}$ denote precision, recall, and F1 on the <code>mix_required</code> class. AUPRC values on the <code>comparison</code> and <code>inference</code> slices are computed on too few <code>mix_required</code> positives to be informative and should be read as a description of these particular test rows rather than as a stable estimate. ModernBERT values are the mean over three seeds.	36
B.1	LightRAG, data, and judge configuration used in this thesis. The same generator model and embedding model are used during indexing and at query time. All Gemini calls use a temperature of 0.0.	56
B.2	Router training configuration used in this thesis.	57

List of acronyms and abbreviations

AUPRC	area under the precision-recall curve
AUROC	area under the receiver operating characteristic curve
BERT	Bidirectional Encoder Representations from Transformers
GPU	Graphics Processing Unit
LLM	large language model
LLM-as-a-Judge	large language model as a judge
RAG	Retrieval-Augmented Generation
SDG	Sustainable Development Goal
TF-IDF	term frequency-inverse document frequency
UN	United Nations

Chapter 1

Introduction

This chapter introduces the thesis topic and motivates the problem studied in this thesis project. Section 1.1 gives the necessary background in retrieval-augmented question answering and the cost of different retrieval modes. Section 1.2 defines the problem and research question. Section 1.3 states the purpose of the work, Section 1.4 presents the goals, Section 1.5 outlines the research methodology, Section 1.6 defines the delimitations, and Section 1.7 describes the structure of the rest of the thesis.

1.1 Background

Retrieval-Augmented Generation (RAG) has become a common architecture for grounding **large language model (LLM)** outputs in external documents, especially when the relevant information is private, domain-specific, or changes after model pretraining [1]. A typical **RAG** system retrieves text passages from a corpus and conditions the generator on those passages, making the language model less dependent on memorized knowledge.

The simplest retrieval layer is vector similarity search over document chunks. It is efficient and often effective for local factual questions, but it can struggle when a query depends on linking evidence across documents, resolving relationships between entities, or comparing distributed attributes. Multi-hop question answering benchmarks are designed around exactly this regime, in which the answer can only be derived by chaining several supporting facts together. Graph-enhanced **RAG** systems address this by introducing structured representations of entities, relations, or higher-level document organization. Recent systems such as GraphRAG and LightRAG show how graph-based structure can support retrieval beyond vector similarity chunks

[2, 3].

Richer retrieval is not free, though. Graph construction, graph traversal, larger context assembly, and additional language-model calls all add to the cost of retrieval. At Ericsson, where internal technical repositories are large and fragmented, graph-enhanced retrieval may improve answer quality for difficult questions, but applying it to every query can waste computational power on questions that cheaper vector retrieval already handles [4, 5].

This tension motivates adaptive retrieval. Self-RAG and Adaptive-RAG argue that retrieval behavior should depend on the query rather than follow a fixed policy [6, 4]. The same cost-performance reasoning appears in *model routing*, where a router decides for each query whether a powerful but computationally expensive language model is needed or whether a cheaper, smaller one would be sufficient [5].

The question studied here is narrower than general model routing. Instead of routing between different language models, the project routes between retrieval modes inside a fixed Hybrid RAG setting. The low-cost path is LightRAG’s *Naive* mode, which uses only vector retrieval, and the high-cost path is LightRAG’s *Mix* mode, which combines vector retrieval with graph-enhanced retrieval. The routing layer is intentionally lightweight, because if the router itself requires a computationally expensive model, much of the intended efficiency gain is lost.

1.2 Research problem

The problem approached in this thesis is whether the choice between a cheap vector-based retrieval path and a more expensive graph-enhanced retrieval path in Hybrid RAG can be made before retrieval runs, from the query text alone. In LightRAG, this is operationalized as a choice between *Naive* retrieval and *Mix* retrieval. Prior work motivates both graph-based RAG and adaptive retrieval [2, 3, 6, 4], but does not quantify how well this retrieval-mode choice can be predicted in advance inside a fixed Hybrid RAG framework.

1.2.1 Problem definition

The research problem is to decide, for a given query, whether LightRAG’s cheaper *Naive* mode is judged sufficient or whether the more expensive *Mix* mode is judged beneficial under the experimental pipeline.

The scientific issue is whether this judged retrieval need is predictable from the query alone. If it depends mostly on what the retriever surfaces

after retrieval has already run, a router that sees only the query text cannot anticipate it. If it depends at least partly on how the question is phrased, a supervised classifier may learn a useful decision boundary.

The distinction studied here is operational rather than causal. A query labeled `mix_required` means that *Mix* was judged correct when *Naive* was not under the chosen LightRAG configuration and judge prompt. It does not prove that graph traversal alone caused the improvement, because *Mix* also changes context assembly and retrieval breadth.

1.2.2 Research question

Within a fixed Hybrid **RAG** setting, how well can a lightweight query-only classifier predict a judge-derived routing label for the minimum sufficient retrieval mode, and how does using that classifier as an offline routing policy affect routed cost relative to static *Naive* and *Mix* baselines?

The research question is further split into the following sub-questions.

- How well does a fine-tuned encoder-only classifier predict the judge-derived routing label compared to a **term frequency-inverse document frequency (TF-IDF)** logistic-regression baseline?
- When classifier scores are turned into a thresholded routing policy, how much routed execution time and estimated model-token cost are saved relative to always-*Mix* at the selected routing-label operating point?
- How much of the routing signal is explained by 2WikiMultihopQA question type, and how much remains predictable from query text alone?

1.3 Purpose

The purpose is to investigate whether retrieval-mode selection in a Hybrid **RAG** system can be made adaptive without moving the routing decision into another expensive model call. The more expensive *Mix* mode should be used when it is judged beneficial, but avoided when *Naive* retrieval is already judged sufficient.

The question is whether the *Naive*-sufficient versus *Mix*-beneficial distinction is predictable enough from the query text that a lightweight classifier can route each query to an appropriate retrieval mode. The contribution is therefore empirical. The thesis tests whether this operational

retrieval-mode distinction can be learned in a fixed LightRAG setting, and how such a router changes routed cost relative to static policies.

1.4 Goals

The main goal is to design and evaluate a learned routing layer that predicts whether a query should use LightRAG’s *Naive* or *Mix* retrieval mode. Four concrete objectives structure the work.

1. Implement a reproducible pipeline for preparing samples from a multi-hop QA benchmark, indexing them in LightRAG, and executing both *Naive* and *Mix* retrieval modes.
2. Construct a supervised routing dataset by labeling each query according to the minimum judged sufficient retrieval mode, using an **large language model as a judge (LLM-as-a-Judge)** to compare retrieval outputs against benchmark answers.
3. Train lightweight query-only routers, including a ModernBERT-based classifier and a classical **TF-IDF** logistic-regression baseline, to predict the *Naive*-sufficient versus *Mix*-beneficial routing label.
4. Evaluate the routers both as held-out classifiers, using routing-label metrics such as **area under the precision-recall curve (AUPRC)** and F1 on the *Mix*-beneficial class, and as thresholded offline routing policies measured against always-*Naive* and always-*Mix* baselines on mean routed execution time and estimated mean routed model-token usage.

1.5 Research methodology

This thesis is a quantitative empirical study on a public multi-hop question-answering benchmark, as is common for evaluating retrieval and routing methods. The predictability of retrieval-mode selection from the query text is evaluated by training query-only routers on judge-derived routing labels and obtaining classification metrics on held-out data. These include **AUPRC**, balanced routing-label accuracy, and F1 on the minority class. The same routers are additionally evaluated as thresholded offline routing policies and compared against static baseline policies along routed-cost metrics. Bootstrap confidence intervals are used to estimate the uncertainty of these metrics.

Comparisons between the learned routers and the static baselines are then used to answer the research question.

The reported classification metrics measure agreement with the judge-derived routing label, not direct final-answer accuracy of the selected retrieval path. In particular, routing a *Naive*-sufficient query to *Mix* is counted as a routing-label error because the cheaper mode was sufficient, even though the *Mix* answer may still be correct. This distinction is important for interpreting the static always-*Mix* baseline and the routed-cost results in Chapter 4.

1.6 Delimitations

The scope is deliberately limited to query-only retrieval-mode routing. The router receives the query text as input and does not inspect retrieved documents, intermediate graph state, answer drafts, or model uncertainty after generation. This makes the routing problem cheaper and cleaner, but it means the project does not evaluate routers that adapt after retrieval has already begun.

The retrieval setting is LightRAG with two modes, *Naive* and *Mix*. These operationalize the broader vector-versus-graph distinction, but results should not be assumed to transfer automatically to every Hybrid RAG framework.

The primary benchmark is 2WikiMultihopQA [7], chosen because it is public, provides supporting evidence, and makes the experiment reproducible without depending on confidential data. Ericsson documentation motivates the industrial problem but is not the core benchmark.

Routing labels come from LLM-as-a-Judge and are treated as judged supervision rather than absolute ground truth. Cases where neither retrieval mode produces a correct answer are excluded from router training.

The offline policy evaluation does not re-judge newly generated answers, because it reuses precomputed *Naive* and *Mix* outputs. Unless stated otherwise, reported accuracy values refer to routing-label accuracy. A false-positive *Mix* decision is therefore counted as a routing-label error because *Naive* was judged sufficient, even though the *Mix* answer may still be correct. Conversely, a false-negative *Mix* decision is an under-routing error, because the router selects *Naive* on a query where *Mix* was judged necessary under the labeling protocol.

1.7 Structure of the thesis

Chapter 2 presents the technical and scientific background on RAG, graph-enhanced retrieval, adaptive retrieval, query routing, ModernBERT, and related work. Chapter 3 describes the experimental method, including data preparation, LightRAG execution, label generation, model training, and evaluation design. Chapter 4 describes the implementation, presents the held-out classification and offline routed-cost experiments, reports the results, and discusses what they imply about the predictability of retrieval-mode selection. Chapter 5 concludes with answers to the research question and suggestions for future work.

Chapter 2

Background

This chapter introduces the technical and scientific concepts that the rest of the thesis builds on. Section 2.1 describes RAG, why it is needed for grounding large language model outputs, and how the components fit together. Section 2.2 contrasts vector retrieval with graph-enhanced retrieval, including the two LightRAG retrieval modes used in this thesis. Section 2.3 describes adaptive retrieval and query routing. Section 2.4 introduces the language models and text classifiers used by the router. Section 2.5 positions the thesis in relation to prior work.

2.1 Retrieval-augmented generation

Large language models encode an enormous amount of general knowledge in their parameters, but they have two limitations relevant to industrial question answering. They cannot access information that was not part of their training data, which makes them unable to answer questions over private or company-internal documents, and they have a tendency to hallucinate, generating fluent but factually incorrect answers when asked about content outside their training distribution [1]. RAG addresses both problems by retrieving relevant passages from an external corpus at query time and conditioning the language model on those passages, which both gives the model access to information beyond its training data and lets it ground its answers in cite-able sources [8, 1].

The original RAG formulation by Lewis et al. [8] treats retrieval and generation jointly as a latent-variable problem. Given a query q , a retriever returns a set of documents $\mathcal{D}_k(q)$, and the generator produces an answer a by

marginalizing over which document was retrieved, as in

$$p(a | q) = \sum_{d \in \mathcal{D}_k(q)} p_\theta(a | q, d) p_\eta(d | q). \quad (2.1)$$

Here $p_\eta(d | q)$ is the retriever, parameterized by η , which assigns a probability to each candidate document given the query, and $p_\theta(a | q, d)$ is the generator, parameterized by θ , which produces the answer given the query and a retrieved document. Equation 2.1 is not how every modern **RAG** system is implemented in practice, but it makes one point that matters for this thesis explicit. Which documents the retrieval system surfaces directly determines what the generator can answer.

For industrial question answering the motivation is concrete. At Ericsson, internal technical documents cannot be assumed to appear in any public language-model pretraining corpus, so a **RAG** system is needed to ground answers in company-controlled knowledge. The same is true of many other domain-specific settings, which is why **RAG** has become a standard architecture for grounded question answering over private corpora [1].

Many real questions cannot be answered from a single passage. Multi-hop question answering benchmarks such as HotpotQA, 2WikiMultihopQA, and MuSiQue are constructed around questions whose answers require combining information from multiple supporting documents [9, 7, 10]. For example, a question of the form “*Who is the spouse of the director of film X?*” requires the system first to identify the director of X, then to look up that person, and finally to retrieve their spouse. A retriever that scores each document independently against the query may surface only part of the chain.

2.2 Graph-enhanced retrieval

Most practical **RAG** systems start with vector retrieval. Documents are split into chunks, each chunk is encoded into a dense vector by an embedding model, and at query time the chunks whose vectors are closest in the embedding space to the query’s vector are returned to the generator [8, 11]. This is fast and works well when the answer is contained inside a single passage that resembles the query.

Vector retrieval is weaker when the required evidence is split across multiple documents or depends on relationships between entities. Closeness in the embedding space measures local textual similarity, not the structure that links documents together. As a result, matching the query against individual

chunks can surface passages that are each relevant on their own but together still miss the full answer chain.

2.2.1 Graph-based RAG systems

Graph-based **RAG** systems address this limitation by constructing a graph over the corpus before any query is asked. Nodes correspond to entities, chunks, or higher-level summaries, and edges correspond to relations between them. At query time the system can then retrieve evidence by traversing this graph rather than only matching the query against individual chunks.

GraphRAG [2] extracts entities and their relations from each chunk using a language model, builds a knowledge graph over the entire corpus, and groups closely related entities into communities. Each community is summarized into a higher-level description, which can be used to answer queries that span large parts of the corpus. LightRAG [3] follows a similar two-level idea, but is designed to be faster and easier to deploy. It also extracts an entity-relation graph during indexing and uses it together with vector chunks at retrieval time, and it exposes several retrieval modes that vary in how heavily they use this graph. The key practical difference for this thesis is that LightRAG explicitly exposes a cheap vector-only retrieval mode and a more expensive graph-enhanced mode in the same framework, which makes it possible to compare the two modes on identical inputs.

Graph-enhanced retrieval is significantly more computationally expensive than pure vector retrieval. Building the graph requires many language-model calls during indexing for entity and relation extraction, and answering a query involves traversing the graph, assembling a larger context, and often additional language-model calls. Microsoft’s cost discussion for GraphRAG identifies these repeated language-model calls as the dominant cost driver [2, 12]. Graph-enhanced retrieval is therefore best treated as an expensive capability that is worth invoking when the question benefits from it, and worth avoiding otherwise.

2.2.2 LightRAG *Naive* and *Mix* modes

LightRAG exposes several retrieval modes, two of which are used as the routed paths in this thesis [3]. The *Naive* mode retrieves the top- k text chunks for the query by vector similarity and passes them directly to the generator, ignoring the graph. The *Mix* mode is hybrid. It retrieves vector chunks as in *Naive* but also retrieves entities and relations from the entity-relation graph, then

assembles all of this into the generator’s context. The two modes therefore share the same underlying corpus and generator but differ in how much graph structure they exploit, which makes them a natural pair of routed retrieval paths.

Figure 2.1 illustrates the difference with a stylized multi-hop example. The *Naive* mode retrieves chunks similar to the query and can fail when no single chunk contains both required hops. The *Mix* mode follows entity-relation edges in the graph and can connect the two hops even when they live in separate documents.

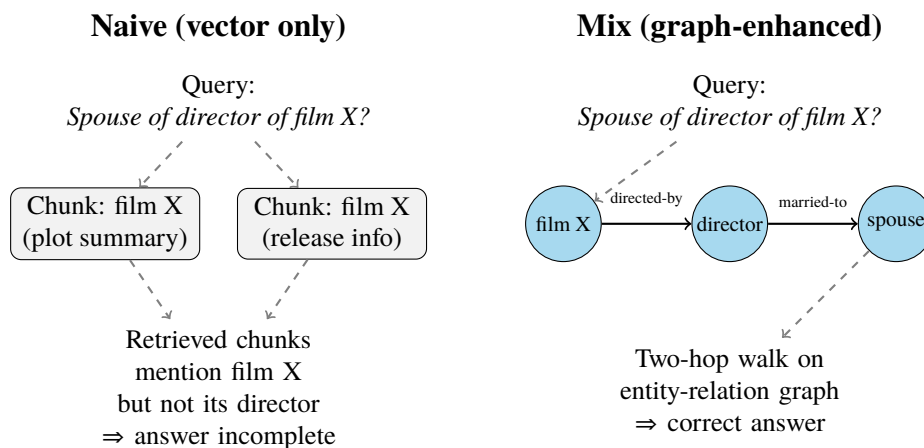


Figure 2.1: Stylized illustration of vector retrieval (*Naive*) versus graph-enhanced retrieval (*Mix*) on a two-hop question. Vector retrieval matches chunks individually against the query, while the graph-enhanced mode can follow entity-relation edges that connect facts living in different documents.

2.3 Adaptive retrieval and query routing

Adaptive retrieval rests on a simple observation. Not every query benefits from the same retrieval strategy. Self-RAG [6] fine-tunes the generator itself to emit reflection tokens that decide whether retrieval is necessary, what to retrieve, and whether the generated answer is supported by the evidence, which lets the system skip retrieval entirely on questions it is confident about. Adaptive-RAG [4] takes a slightly different approach. It trains a small classifier to estimate the complexity of a question and uses that estimate to select between progressively more expensive retrieval-augmented reasoning strategies, ranging from no retrieval through single-hop retrieval to iterative retrieval. Both works support

the premise that retrieval behavior should depend on the query, and Adaptive-RAG in particular shows that a small upstream classifier can drive the strategy choice.

Model routing frames the same idea more generally as a cost–quality decision [5]. In a multi-model setting, the system has access to several language models with different cost and accuracy profiles, and a router decides on a per-question basis whether to use a powerful but expensive model or a cheaper, weaker one. RouterBench [5] is a benchmark and evaluation suite for this problem and reports that even simple routers can recover a large fraction of the strongest model’s accuracy while spending only a fraction of its cost. Although RouterBench concerns routing between language models rather than between retrieval modes, the abstraction transfers directly. A router observes an input and chooses among execution paths with different cost and expected performance before the expensive computation happens.

This thesis applies that routing abstraction inside the retrieval layer, selecting between two retrieval modes within one Hybrid RAG framework. Figure 2.2 illustrates the routing decision. Adaptive-RAG and similar work likewise make their routing decision before retrieval runs, on the query alone [4, 5].

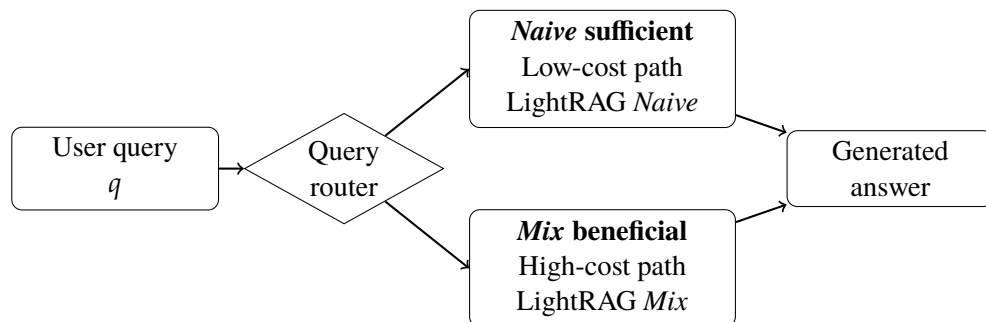


Figure 2.2: Query routing between LightRAG’s cheap *Naive* path and its more expensive *Mix* path. The router observes only the query and selects a retrieval mode before retrieval runs. The labels are operational. *Mix* beneficial means that *Mix* was judged beneficial relative to *Naive* under the experimental pipeline.

2.4 Language models and text classifiers

The routing layer is implemented as a text classifier over the query. This section describes the two families of classifier compared in this thesis. The

first is a transformer-based encoder model, ModernBERT. The second is a classical **TF-IDF** logistic-regression baseline.

2.4.1 Encoder-only transformers and ModernBERT

The transformer architecture [13] introduced self-attention as a general mechanism for contextualized text representations. Encoder-only models such as **Bidirectional Encoder Representations from Transformers (BERT)** are trained on masked language modeling and produce a contextualized vector per input token, which can be pooled or fed to a small classification head to produce a class label [14]. Decoder-only models such as the GPT family are instead trained to predict the next token and are more naturally used as generative language models.

Both families can in principle be used for classification, but encoder-only models remain attractive when the output is a class label and inference efficiency matters. A recent comparison by Benayas, Sicilia, and Mora-Cantalops [15] finds that encoder-only models match or exceed much larger decoder-only models on intent classification and sentiment analysis while being orders of magnitude smaller, which is consistent with the design goal of a cheap routing layer.

ModernBERT [16] is a recent encoder model that updates the **BERT** architecture with modern training data, longer context, and several efficiency improvements. The authors report that ModernBERT-base matches or exceeds the classification quality of much larger models while remaining cheap enough to fine-tune on a single **Graphics Processing Unit (GPU)**, which makes it a natural choice for a query-only router that has to remain inexpensive relative to the retrieval modes it routes between.

2.4.2 TF-IDF logistic regression

The classical baseline used in this thesis is a logistic regression over **TF-IDF** features [17]. Each query is represented by a sparse vector whose entries are term frequencies weighted by the inverse document frequency of the term in the training corpus, which downweights very common words. A logistic regression then maps this sparse feature vector to a probability that the query is *Mix*-beneficial under the judged setup. The baseline matters because the distinction between *Naive*-sufficient and *Mix*-beneficial queries may be partly lexical. Multi-hop questions often have characteristic surface patterns. Nested relations like “*the spouse of the director of X*” signal a chained lookup,

and comparisons between two entities signal that evidence must come from multiple sources. A **TF-IDF** classifier can pick up such patterns from word frequencies alone, so it sets a meaningful floor for how much a pretrained transformer encoder needs to add.

2.5 Related work

The most closely related prior work falls into three groups. The first is graph-based **RAG** systems. The second is adaptive-retrieval work that uses an upstream classifier. The third is general model routing.

Edge et al. [2] introduce GraphRAG, and Guo et al. [3] introduce LightRAG. Both systems demonstrate that graph-enhanced retrieval can add structure beyond local vector similarity and improve answer quality on questions that require linking facts across documents. They also document the additional cost of doing so. Neither work addresses the follow-up question of how the system should decide which retrieval mode to use for a given query. The work in this thesis treats that selection as a supervised prediction problem.

Jeong et al. [4] present Adaptive-RAG, the closest precedent on the methodological side. They train a small classifier on the question to estimate complexity and use that estimate to select between progressively more expensive retrieval-augmented reasoning strategies, ranging from no retrieval to iterative retrieval. The classifier is trained on labels derived from how well different strategies answer the question on a training set, which is conceptually similar to the judge-derived labels used here. Adaptive-RAG and this thesis differ in two respects. Adaptive-RAG selects across these reasoning strategies at the language-model level, whereas this thesis selects between LightRAG’s *Naive* and *Mix* retrieval modes inside one fixed framework. The classifier here is also intentionally restricted to a lightweight encoder so the routing cost remains negligible. Asai et al. [6] achieve a similar adaptive behavior with Self-RAG by training the generator itself to emit reflection tokens, which is a more invasive change and does not give an explicit pre-retrieval routing decision.

Hu et al. [5] formalize routing between language models as a cost and quality trade-off, and provide RouterBench as a benchmark for it. Although RouterBench routes between language models rather than between retrieval modes, the framing carries over directly. The trade-off plots used in Chapter 4 follow the same logic. Each operating point of the learned router can be compared to static baselines, and the relevant question is whether the router achieves useful routing-label performance at substantially lower routed cost

than the always-expensive policy.

A difference between model-routing benchmarks and the setting studied here is that a wrong expensive-route decision does not necessarily reduce answer quality. If a query is *Naive*-sufficient, routing it to *Mix* is a cost error rather than necessarily an answer error. This makes retrieval-mode routing slightly different from ordinary binary classification. False negatives risk answer failure by routing a *Mix*-beneficial query to *Naive*, while false positives mainly waste computation by routing a *Naive*-sufficient query to *Mix*. The evaluation in this thesis therefore reports routing-label metrics separately from routed cost.

What is missing from this literature is a focused empirical study of the choice between LightRAG’s *Naive* and *Mix* retrieval modes inside one Hybrid **RAG** framework, treated as a supervised, query-only routing problem with judge-derived labels. That is the gap this thesis fills.

Chapter 3

Methods

This chapter describes the data, the routing pipeline, the router models, and the evaluation procedure used to study query-only retrieval-mode routing. Section 3.1 describes the benchmark data and how it is preprocessed and indexed. Section 3.2 describes the routing-label pipeline that turns raw benchmark questions into a supervised routing dataset. Section 3.3 describes the resulting binary routing dataset and the held-out splits (the validation and test splits, neither of which is used to fit any model parameters) used for evaluation. Section 3.4 describes the two router models, a ModernBERT-based classifier and a **TF-IDF** logistic-regression baseline. Section 3.5 describes the held-out classification metrics and the offline routing-policy evaluation.

3.1 Data

3.1.1 2WikiMultihopQA

The benchmark used in this thesis is 2WikiMultihopQA [7], a public multi-hop question answering dataset constructed from Wikipedia. Each sample contains a question, a gold answer, a set of context documents drawn from Wikipedia, and a set of supporting facts that mark which sentences in the context are required to derive the answer. Questions are annotated with one of four types that describe the reasoning structure. These types are `comparison`, `compositional`, `bridge_comparison`, and `inference`.

2WikiMultihopQA is chosen for three reasons. It is public, which makes the experiment reproducible. It is multi-hop by construction, so the answer cannot in general be read off a single passage, which is the setting in which

graph-enhanced retrieval is expected to differ most from vector retrieval. The supporting-fact annotations provide traceable evidence metadata, although this thesis uses them mainly for dataset traceability rather than a full supporting-fact analysis. Other multi-hop benchmarks such as HotpotQA and MuSiQue [9, 10] share this motivation. 2WikiMultihopQA was preferred because, in preliminary testing across the three benchmarks under LightRAG’s dual-mode execution, it produced the most balanced `mix_required` versus `naive_enough` distribution. The other two benchmarks gave more skewed distributions, which would have made supervised routing harder to evaluate.

3.1.2 Preprocessing and indexing

Each sample is converted into a structured record containing a stable sample identifier, the question, the gold answer, the question type, the context documents, and the supporting facts. The mapping between sample identifier and these fields is preserved throughout the pipeline so that every later artifact (retrieval output, judge label, router prediction) can be traced back to the original sample.

Context documents from these structured records are then inserted into LightRAG [3], which during indexing builds both a vector index over chunked text and an entity-relation graph derived from the same chunks. The indexing pass is incremental and idempotent. Previously inserted documents are recognized and skipped, which makes it possible to resume an interrupted run without rebuilding the index from scratch. Once the index has been built, it is treated as a fixed part of the experimental setup. Stochasticity in graph construction (entity extraction, relation extraction, summarization) is therefore baked into the index and does not introduce additional variance during the dual-mode retrieval experiments.

3.2 Routing-label pipeline

Figure 3.1 shows the end-to-end pipeline. Starting from 2WikiMultihopQA samples, the corpus is indexed in LightRAG, every sample is executed with both *Naive* and *Mix* retrieval, an **LLM-as-a-Judge** assigns a routing class to each query, and the resulting binary labels are used to train and evaluate query-only routers.

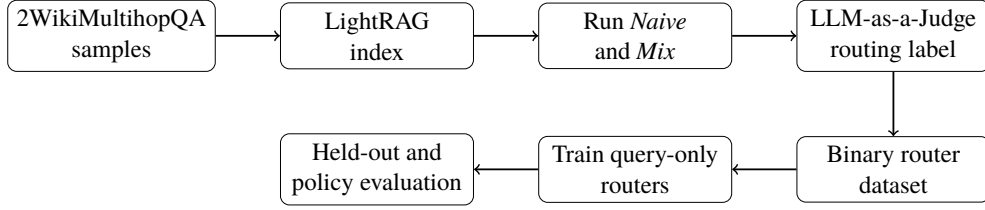


Figure 3.1: End-to-end pipeline from 2WikiMultihopQA samples to router evaluation. Each step writes its output to disk so the remaining steps can be re-run without repeating expensive earlier steps.

3.2.1 Dual-mode retrieval

For every benchmark question q_i , the indexed corpus is queried with both LightRAG retrieval modes, giving

$$r_i^N = R_N(q_i), \quad r_i^M = R_M(q_i), \quad (3.1)$$

where R_N denotes LightRAG *Naive* (the cheap vector path) and R_M denotes LightRAG *Mix* (the graph-enhanced path). The recorded artifacts for each query are the two generated answers r_i^N and r_i^M , the measured execution time t_i^N and t_i^M for each mode, and the number of context documents and supporting facts associated with the sample.

Two cost dimensions are reported. Execution time is logged per query for every query in the dataset. Model-token usage is additionally measured on an instrumented subset of $N = 150$ queries spanning all four 2WikiMultihopQA question types, with token counts taken from Gemini API `usage_metadata` when available and otherwise estimated with the configured tokenizer. The instrumented subset is used to estimate the per-mode mean model-token consumption, denoted $\bar{u}^N$ for the *Naive* retrieval mode and $\bar{u}^M$ for the *Mix* retrieval mode. These per-mode means are then applied to the held-out test split for the routed-cost analysis in Section 3.5.2. The routed token figures are therefore estimated from per-mode token means rather than measured per query for every test example. Reporting both cost dimensions matters because execution time is partly sensitive to network latency in API-based LightRAG calls, while model-token usage corresponds more directly to per-query operational cost in production deployments. The generator model, judge model, embedding model, LightRAG version, retrieval parameters, token logging method, and judge prompt are listed in Appendix B.

3.2.2 LLM-as-a-Judge labeling

The 2WikiMultihopQA benchmark provides gold answers but does not provide routing labels. Routing labels are constructed by running an **LLM-as-a-Judge** [18] that compares each query’s two generated answers against the benchmark answer. For each query the judge returns one of three classes. The class `naive_enough` is assigned when r_i^N matches the gold answer, so the cheap mode was judged sufficient. The class `mix_required` is assigned when r_i^N does not match the gold answer but r_i^M does, so *Mix* was judged beneficial relative to *Naive* under the experimental pipeline. The class `none_enough` is assigned when neither mode produced an answer that matches the gold answer. This produces the binary routing label

$$y_i = \begin{cases} 0, & \text{judge class is } \text{naive_enough}, \\ 1, & \text{judge class is } \text{mix_required}, \end{cases} \quad (3.2)$$

with `none_enough` cases excluded from the binary router training set, since for these queries neither retrieval mode succeeds, so the question reflects an answer-quality failure that is outside the scope of routing.

The label name `mix_required` is retained as a compact code label, but it should be interpreted operationally as *Mix*-beneficial under this judged LightRAG setup. It does not prove that graph traversal alone caused the improvement, because *Mix* also changes context assembly, retrieval breadth, and the evidence exposed to the generator.

The **LLM-as-a-Judge** is treated as judged supervision rather than ground truth. The risk of judge error is mitigated in three ways. First, the judge sees the gold answer in addition to the candidate answers, which turns the task into a constrained comparison rather than open-ended quality scoring [18]. Second, ambiguous `none_enough` cases are excluded from router training rather than forced into one of the two binary classes. Third, the same judge model and prompt are used across the entire dataset, which keeps the labeling protocol internally consistent.

3.3 Binary routing dataset

Before the binary dataset is analyzed by split, Table 3.1 summarizes how the raw dual-mode runs are filtered into the final binary routing dataset.

After the dual-mode execution and judge labeling, the binary routing dataset contains 1 286 judged queries with a routing class of either

Table 3.1: Label-flow summary from raw benchmark samples to the final binary routing dataset. The binary router is trained only on `naive_enough` and `mix_required` rows. The `mix_required` label is operational and means *Mix*-beneficial under the judged LightRAG setup.

Stage	Count
Sampled 2WikiMultihopQA queryable samples	2 279
Queries successfully run with both <i>Naive</i> and <i>Mix</i>	2 000
Judge produced usable label	2 000
<code>naive_enough</code>	959
<code>mix_required</code> (<i>Mix</i> -beneficial)	327
<code>none_enough</code> (excluded from binary dataset)	714
Final binary routing dataset	1 286

`naive_enough` or `mix_required`. Of these, 959 (74.6 %) are labeled `naive_enough` and 327 (25.4 %) are labeled `mix_required`. The class distribution is therefore imbalanced, which is taken into account during training and during reporting (Section 3.5).

The dataset is split into 900 training examples, 193 validation examples, and 193 test examples (approximately 70/15/15), jointly stratified by question type and routing label so that both the class proportions and the per-question-type proportions appear in all three splits in the same proportion as in the overall dataset. The validation split is held out from training and is used only for early stopping and threshold selection. The test split is held out from both training and threshold selection and is used only for final reported metrics.

Table 3.2 summarizes the routing dataset by split and 2WikiMultihopQA question type. The class proportions are preserved across splits by construction, but the distribution of `mix_required` labels is uneven across question types. `Compositional` questions are by far the most likely to be judged *Mix*-beneficial, while `comparison` questions are usually answered well enough by the cheap mode.

3.4 Router models

Both router models take only the question text q_i as input and output a score $s_\phi(q_i) \in [0, 1]$, where ϕ denotes the model parameters. The score is interpreted as $P_\phi(y_i = 1 \mid q_i)$, the model’s estimated probability that the binary routing label y_i is 1, i.e., that *Mix* is judged beneficial for query q_i .

Table 3.2: Routing dataset summary by split and 2WikiMultihopQA question type. Each cell shows the total number of judged binary examples followed by the count of those examples that are labeled `mix_required`.

Split	Total	comparison	compositional	bridge comp.	inference
Train	900	415 (8 mix)	305 (178 mix)	155 (37 mix)	25 (6 mix)
Validation	193	89 (2 mix)	66 (38 mix)	33 (8 mix)	5 (1 mix)
Test	193	90 (2 mix)	65 (38 mix)	33 (8 mix)	5 (1 mix)
Overall	1 286	594 (12 mix)	436 (254 mix)	221 (53 mix)	35 (8 mix)

under the experimental pipeline.

3.4.1 TF-IDF logistic-regression baseline

The baseline router is a logistic regression trained on **TF-IDF** features of the question. The pipeline first builds a vocabulary from the training questions, where each token is a unigram or bigram after lowercasing and basic English stop-word removal. Each question is then represented as a sparse vector $\mathbf{x}_i \in \mathbb{R}^V$ whose entries are the term frequency of each vocabulary term multiplied by the inverse document frequency of that term in the training corpus. The logistic-regression head produces the router score as

$$s_\phi(q_i) = \sigma(\mathbf{w}^\top \mathbf{x}_i + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (3.3)$$

where $\mathbf{w} \in \mathbb{R}^V$ and $b \in \mathbb{R}$ are learned by minimizing the L2-regularized binary cross-entropy loss with balanced class weights, so that the minority `mix_required` class is not ignored.

The baseline is intentionally simple. It has no pretrained semantic representations and can only exploit lexical patterns in the question. The role of this baseline is to indicate how much of the routing signal is captured by surface lexical cues, which calibrates how much improvement a transformer encoder can plausibly add.

3.4.2 ModernBERT router

The transformer router is a ModernBERT-base [16] encoder with a binary classification head. The question is tokenized and encoded by ModernBERT into a sequence of contextualized hidden states. The pooled representation

$h_i \in \mathbb{R}^d$ is then mapped to two logits by the classification head,

$$z_i = Wh_i + b, \quad W \in \mathbb{R}^{2 \times d}, \quad b \in \mathbb{R}^2, \quad (3.4)$$

where $z_{i,0}$ corresponds to `naive_enough` and $z_{i,1}$ to `mix_required`. The router score is the softmax probability of the `mix_required` class, given by

$$s_\phi(q_i) = \frac{e^{z_{i,1}}}{e^{z_{i,0}} + e^{z_{i,1}}}, \quad (3.5)$$

where e is Euler’s number. Equivalently, the same probability can be written using a single logit and the sigmoid function, $s_\phi(q_i) = \sigma(z_{i,1} - z_{i,0})$. The two-logit parameterization used here matches the default Hugging Face classification-head format for binary classification with `num_labels=2`.

The model is fine-tuned by minimizing class-balanced cross-entropy on the judged routing labels,

$$\mathcal{L}(\phi) = -\frac{1}{n} \sum_{i=1}^n \alpha_{y_i} \log P_\phi(y_i | q_i), \quad (3.6)$$

where α_{y_i} is the inverse class-frequency weight for the label of example i , so that the minority `mix_required` class is not down-weighted by the imbalanced training distribution. Optimization uses AdamW [19] with a linear warm-up and linear decay schedule. The best checkpoint is selected on the validation split using the F1 score on the `mix_required` class, and the chosen checkpoint is the one used for all reported test metrics. Hyperparameters are listed in Chapter 4.

3.5 Evaluation

Evaluation separates routing-label prediction from routed-cost analysis. Held-out classification metrics measure agreement with the judge-derived routing label. They do not directly measure final answer accuracy of the selected retrieval path. In particular, routing a `naive_enough` query to *Mix* is counted as a routing-label error because the cheaper mode was sufficient, but the *Mix* answer may still be correct. Conversely, routing a `mix_required` query to *Naive* is both a routing-label error and an answer-risk error under the label definition.

Offline policy evaluation measures what happens when the router’s predictions are used to select between the precomputed *Naive* and *Mix* retrieval

outputs. Both evaluations use the held-out test split. The validation split is used only for early stopping and for selecting an operating threshold.

3.5.1 Held-out classification metrics

Because the binary class distribution is imbalanced, the primary threshold-independent metric is the **AUPRC** on the `mix_required` class. **AUPRC** measures how well the router ranks `mix_required` queries above `naive_enough` queries and is more informative than **area under the receiver operating characteristic curve (AUROC)** when the positive class is in the minority [20]. **AUROC** is also reported for comparison with prior work.

For thresholded evaluation, the router score $s_\phi(q_i)$ is converted to a class prediction by applying a threshold τ . The thresholded policy is

$$\pi_\tau(q) = \begin{cases} M, & s_\phi(q) \geq \tau, \\ N, & s_\phi(q) < \tau, \end{cases} \quad (3.7)$$

where M denotes *Mix* and N denotes *Naive*. At a chosen threshold the reported classification metrics are routing-label accuracy, balanced routing-label accuracy, precision and recall on the `mix_required` class, and the F1 score

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}. \quad (3.8)$$

The word accuracy is therefore used in the routing-label sense throughout this thesis. It measures whether the policy selected the same mode as the judge-derived binary label and should not be read as final answer accuracy of the routed QA system. Balanced routing-label accuracy is included because the class distribution is imbalanced and plain routing-label accuracy can be high even for a trivial always-*Naive* policy.

Where bracketed confidence intervals are reported, they are computed using a 1 000-iteration bootstrap over the held-out test set. Each bootstrap sample resamples the test rows with replacement and recomputes the corresponding metric, and reported intervals are the 2.5th and 97.5th percentiles of the resulting bootstrap distribution. These intervals capture uncertainty from the finite test set, meaning how the metric might change if a different sample of test queries had been drawn.

ModernBERT is trained with three random seeds. The result tables report the mean and standard deviation across these seeds, which captures variance from initialization and training randomness. Unless explicitly shown

in brackets, the ModernBERT values are therefore seed means and standard deviations rather than bootstrap confidence intervals.

3.5.2 Offline routing policy

The offline policy evaluation reuses the precomputed retrieval outputs and execution times from Section 3.2.1. For each held-out query, both r_i^N and r_i^M are already available, along with their execution times t_i^N and t_i^M . A routing policy π_τ at threshold τ is then simulated row by row. For each query the policy selects either the *Naive* or the *Mix* precomputed result, and aggregate statistics are computed across the entire held-out split.

For each routing policy and each cost type, the routed cost averages the per-query cost of whichever retrieval mode the policy selected. For execution time this gives

$$C(\pi_\tau) = \frac{1}{m} \sum_{i=1}^m [\mathbf{1}\{\pi_\tau(q_i) = N\} t_i^N + \mathbf{1}\{\pi_\tau(q_i) = M\} t_i^M], \quad (3.9)$$

where m is the number of held-out queries. The mean routed model-token cost is computed analogously, using the per-mode means $\bar{u}^N$ and $\bar{u}^M$ from the instrumented subset (Section 3.2.1) in place of the per-query times:

$$\begin{aligned} U(\pi_\tau) &= \frac{1}{m} \sum_{i=1}^m [\mathbf{1}\{\pi_\tau(q_i) = N\} \bar{u}^N + \mathbf{1}\{\pi_\tau(q_i) = M\} \bar{u}^M] \\ &= (1 - r_M) \bar{u}^N + r_M \bar{u}^M, \end{aligned} \quad (3.10)$$

where r_M is the fraction of queries routed to *Mix* at threshold τ . This is an extrapolated mean token cost and is reported alongside the time-based cost in Chapter 4.

The same simulation is run for two static baseline policies for comparison. The first is an always-*Naive* policy, and the second is an always-*Mix* policy. These baselines bracket what a static retrieval strategy can achieve on the test set without any per-query decision and serve as the reference points against which the learned router’s routing-label and routed-cost trade-off is reported.

In addition to mean routed cost, the offline policy evaluation reports two routing error rates with different practical meanings. The under-routing rate is the fraction of test queries where the true label is `mix_required` but the

policy routes to *Naive*,

$$\text{under-routing}(\pi_\tau) = \frac{1}{m} \sum_{i=1}^m \mathbf{1}\{y_i = 1, \pi_\tau(q_i) = N\}. \quad (3.11)$$

The over-routing rate is the fraction of test queries where the true label is `naive_enough` but the policy routes to *Mix*,

$$\text{over-routing}(\pi_\tau) = \frac{1}{m} \sum_{i=1}^m \mathbf{1}\{y_i = 0, \pi_\tau(q_i) = M\}. \quad (3.12)$$

Under-routing is the error type most directly associated with answer-quality risk, because the policy selected *Naive* on a query where *Mix* was judged necessary. Over-routing is primarily a cost error, because the policy selected *Mix* on a query where *Naive* was already judged sufficient. This decomposition is useful because a false-positive *Mix* decision is counted as a routing-label error even though the *Mix* answer may still be correct.

Threshold selection uses the validation split only. The threshold reported on test is the one that maximizes F1 on the `mix_required` class on the validation split, and the same threshold is then applied unchanged to test. This prevents the threshold from being tuned on the data it is evaluated on.

Chapter 4

Implementation

This chapter describes the implementation and the experiments. Section 4.1 describes how the two router models were trained. Section 4.2 presents the held-out classification experiment, which evaluates how well each router predicts the judge-derived routing label. Section 4.3 presents the offline routed-cost experiment, which evaluates each router as a thresholded routing policy against the static always-*Naive* and always-*Mix* baselines. Section 4.4 reports the results of both experiments. Section 4.5 discusses what the results imply about the predictability of retrieval-mode selection, where the router fails, and what the practical routing-label and routed-cost trade-off looks like.

4.1 Model training

Both routers are trained on the 900-example training split of the binary routing dataset described in Section 3.3. The validation split is used only for early stopping (ModernBERT) and threshold selection (both routers). The test split is used only for the metrics reported in Section 4.4.

4.1.1 TF-IDF logistic-regression baseline

The baseline is built with scikit-learn [21]. Questions are lowercased and stripped of standard English stop words, then converted into sparse term-frequency vectors with unigram and bigram features (scikit-learn `ngram_range` set to `(1, 2)`), a minimum document frequency of 2, and a vocabulary cap of 20,000 features. The term frequencies are weighted by inverse document frequency [17] learned on the training split. A logistic regression with L_2 regularization (regularization strength $C = 1.0$) is fitted

on top of these features, with class weights set to `balanced` so that the loss on each example is inversely proportional to the frequency of its class in the training data. Optimization uses scikit-learn’s L-BFGS solver [22] with a maximum of 1,000 iterations. Fitting on the 900 training questions takes a few seconds on a single CPU. The same fitted pipeline (vectorizer plus regression head) is then used to score the validation and test splits, producing a probability that each query is *Mix*-beneficial under the judged setup.

4.1.2 ModernBERT router

The ModernBERT router uses the `answerdotai/ModernBERT-base` checkpoint from Hugging Face [16] with a binary classification head. Questions are tokenized with the standard ModernBERT tokenizer and truncated or padded to a maximum length of 128 tokens, which comfortably accommodates every question in the 2WikiMultihopQA splits used here.

Training uses the AdamW optimizer [19] with a learning rate of 2×10^{-5} , a weight decay of 0.01, gradient accumulation of two steps with a per-device batch size of 8 (effective batch size 16), a linear warm-up over the first 10 % of steps followed by linear decay, and gradient clipping at 1.0. The loss is class-balanced cross-entropy with weights α_c inversely proportional to class frequency in the training split, so that the minority `mix_required` class is not down-weighted by the imbalanced training distribution. The model is trained for up to 8 epochs with early stopping based on validation F1 on the `mix_required` class, requiring at least 3 epochs before stopping and using a patience of 3 epochs.

To separate variance from random initialization from variance from finite test data, the ModernBERT router is trained with three different random seeds (7, 13, 42). Each seed independently selects its own best checkpoint on the validation split and its own decision threshold on the validation split. The three runs are reported as a multi-seed mean and standard deviation in Section 4.4. All ModernBERT training and inference was performed on CPU. The model and dataset are small enough that the full fine-tuning run completes in a few hours on a single CPU, which is part of why a lightweight encoder router is attractive as a routing layer.

4.2 Held-out classification experiment

The held-out classification experiment evaluates each router as a binary classifier of judge-derived routing labels on the test split. The router scores

are first evaluated threshold-independently, by computing **AUPRC** on the `mix_required` class and **AUROC**. **AUPRC** is the primary metric because it is sensitive to performance on the minority class, which is the class that matters for the routing decision [20].

For thresholded metrics, the decision threshold is selected on the validation split. The selection rule is the threshold τ^* that maximizes F1 on `mix_required` on the validation split, swept over a fixed grid in $[0.05, 0.95]$. The selected τ^* is then applied unchanged to the test split, where routing-label accuracy, balanced routing-label accuracy, precision, recall, and F1 on `mix_required` are reported. A trivial always-*Naive* predictor is included as a simple baseline because the class imbalance makes plain routing-label accuracy easy to inflate.

A non-deployable question-type diagnostic baseline is also included to measure how much of the routing signal is explained by 2WikiMultihopQA metadata. The rule predicts *Mix* for `compositional` questions and *Naive* for all other question types, because `compositional` is the only question type whose *Mix*-beneficial rate exceeds 50% in the training split. This baseline is not a realistic deployment policy because question-type metadata is unavailable for arbitrary user queries. It is included only as a diagnostic reference for benchmark structure and as a way to separate how much of the routing signal comes from this coarse metadata rule from how much the learned routers add on top of it.

For TF-IDF, bracketed values are 95% bootstrap intervals [23] over 1 000 test-set resamples. For ModernBERT, values are reported as mean $\pm$ standard deviation across three training seeds.

4.3 Offline routed-cost experiment

The offline routed-cost experiment evaluates each router as a thresholded routing policy along two cost dimensions, following the cost–quality routing-evaluation framing introduced by RouterBench [5]. In this section, “performance” refers to routing-label performance, not independently measured final answer accuracy. The precomputed retrieval outputs and per-query execution times for both LightRAG modes are reused so that the simulation does not have to re-run any retrieval. A routing policy π_τ at threshold τ sends each test query either to the *Naive* or to the *Mix* precomputed result. The mean routed execution time is computed as in Equation 3.9. The mean routed model-token cost is computed as in Equation 3.10, using per-mode mean token usage from the instrumented subset described in Section 3.2.1, giving an extrapolated

mean token cost. Under-routing and over-routing rates are computed for each routing policy following Equations 3.11 and 3.12.

For each router, the threshold curve is computed over τ values stepping from 0 to 1 in increments of 0.01, producing a trace of mean routed time and F1 on `mix_required` at each threshold. The same simulation is run for the two static baseline policies, *always-Naive* and *always-Mix*. These two points bracket what a static policy can achieve on the test set and serve as the reference points the learned routers are compared against.

4.4 Results

Tables 4.1 and 4.2 summarize the test-set results. Table 4.1 reports routing-label classification performance, while Table 4.2 reports offline routed cost and policy diagnostics. The separation is important because a false-positive *Mix* decision is a routing-label error but not necessarily an answer error. The policy pays for *Mix* even though *Naive* was already judged sufficient. A false-negative *Mix* decision is more serious, because the policy sends a *Mix*-beneficial query to *Naive*.

ModernBERT achieves higher point estimates than **TF-IDF** on aggregate routing-label metrics, including **AUPRC**, balanced routing-label accuracy, F1 on the *Mix*-beneficial class, and routing-label accuracy. The gain comes mainly from higher precision. ModernBERT sends fewer *Naive*-sufficient queries to *Mix*, so its over-routing rate is lower (0.078 ± 0.019 vs. 0.155 for **TF-IDF**). **TF-IDF** is more conservative on the routing decision in the opposite direction, with higher *Mix* recall and therefore fewer under-routed *Mix*-beneficial queries (0.057 vs. 0.076 ± 0.009 for ModernBERT), but it also routes more unnecessary queries to *Mix*. This difference matters for deployment. Under-routing is the error type most directly associated with answer-quality risk, while over-routing mainly increases cost. ModernBERT therefore behaves as a more selective lower-cost policy, whereas **TF-IDF** behaves as a more conservative policy that spends more tokens to reduce missed *Mix*-beneficial queries.

The type-only diagnostic baseline is surprisingly strong, confirming that benchmark question type explains part of the routing signal. However, this baseline uses metadata that is not available in real deployments. Its role is therefore diagnostic rather than competitive. ModernBERT achieves higher point estimates than this metadata rule on F1 (0.699 vs. 0.667), precision (0.701 vs. 0.585), and routing-label accuracy (0.846 vs. 0.803) while using only the query text. **TF-IDF** matches the diagnostic baseline’s *Mix* recall

(0.776) at lower precision and overall accuracy. The cost picture is also clearer when the two cost dimensions are separated. **TF-IDF** routes at 8 531 mean tokens per query and ModernBERT at 7 138 mean tokens per query, compared with 17 857 tokens for *always-Mix*, a token saving of 52 % for **TF-IDF** and 60 % for ModernBERT relative to the expensive baseline.

Table 4.1: Routing-label classification performance on the held-out test split. P_{mix} , R_{mix} , and $F1_{\text{mix}}$ denote precision, recall, and F1 on the `mix_required` class. Accuracy values are routing-label accuracies, not final answer accuracies. For the constant-score static baselines, **AUROC** equals 0.500 and **AUPRC** equals the test-set `mix_required` base rate of 0.254. The type-only diagnostic baseline is non-deployable because it uses 2WikiMultihopQA metadata rather than only query text. For TF-IDF, bracketed values are 95 % bootstrap intervals over 1 000 test-set resamples. For ModernBERT, values are reported as mean $\pm$ standard deviation across three training seeds.

Policy	AUPRC	AUROC	Bal. rt.-lbl acc.	P_{mix}	R_{mix}	$F1_{\text{mix}}$	Rt.-lbl acc.
<i>Always-Naive</i>	0.254	0.500	0.500	0.000	0.000	0.000	0.746
<i>Always-Mix</i>	0.254	0.500	0.500	0.254	1.000	0.405	0.254
Type-only diagnostic	—	—	0.794	0.585	0.776	0.667	0.803
TF-IDF	0.742 [0.610, 0.853]	0.892 [0.836, 0.936]	0.784	0.559	0.776	0.650 [0.547, 0.740]	0.788 [0.731, 0.845]
ModernBERT	0.769 ± 0.015	0.908 ± 0.011	0.798 ± 0.012	0.701 ± 0.057	0.701 ± 0.042	0.699 ± 0.017	0.846 ± 0.016

Table 4.2: Offline routed-cost and policy diagnostics on the held-out test split. Mean tokens are estimated from per-mode token means measured on the instrumented $N = 150$ query subset (Section 3.2.1). The token saving column reports the reduction in mean tokens relative to the always-*Mix* baseline. Under-routing means routing a `mix_required` query to *Naive*. Over-routing means routing a `naive_enough` query to *Mix*. Under-routing is the error type most directly associated with answer-quality risk, while over-routing is primarily a cost error. ModernBERT values are the mean over three training seeds $\pm$ standard deviation.

Policy	Route-to- <i>Mix</i> fr.	Mean time (s)	Mean tokens	Token saving	Under-rout.	Over-rout.
Always- <i>Naive</i>	0.000	10.95	3 458	80.6 %	0.254	0.000
Always- <i>Mix</i>	1.000	24.42	17 857	0.0 %	0.000	0.746
Type-only diagnostic	0.337	15.59	8 308	53.5 %	0.057	0.140
TF-IDF	0.352	16.98	8 531	52.2 %	0.057	0.155
ModernBERT	0.256 ± 0.027	14.92 ± 0.69	7 138	60.0 %	0.076 ± 0.009	0.078 ± 0.019

Figure 4.1 shows the test-set precision–recall curves on the `mix_required` class for the **TF-IDF** baseline and the three ModernBERT seeds. The base rate (the test-set fraction of `mix_required`) is shown as a horizontal dashed line. The ModernBERT curves dominate the **TF-IDF** curve over most of the recall range, which is consistent with the **AUPRC** numbers in Table 4.1. The **AUROC** curves (Figure 4.2) are closer together, reflecting that **AUROC** is less sensitive to performance on the minority class.

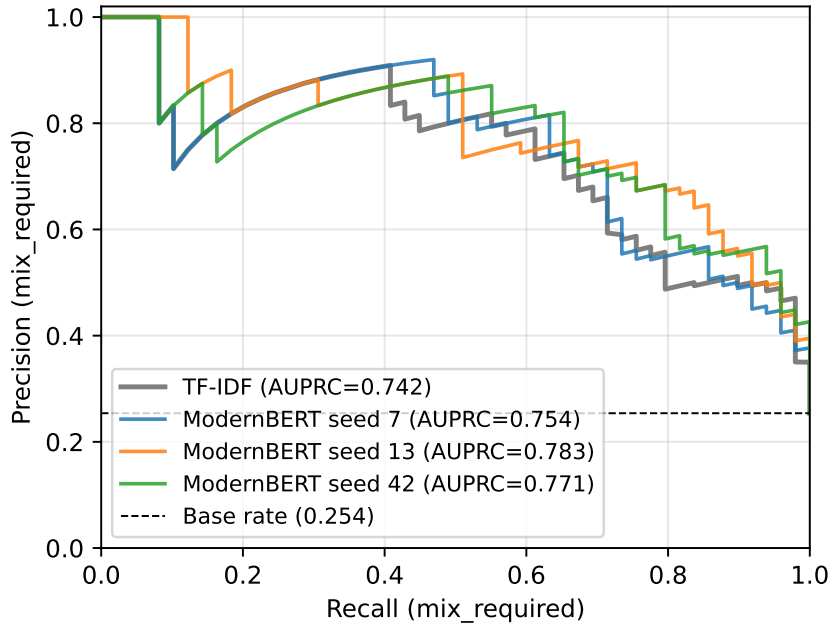


Figure 4.1: Precision–recall curves on the held-out test split for the **TF-IDF** baseline and the three ModernBERT training seeds. The dashed horizontal line is the test-set base rate of `mix_required`.

Figure 4.3 shows how F1 on `mix_required` varies with the decision threshold on the test split, with the validation-selected threshold of each model marked as a dot. The ModernBERT seeds select different thresholds. Seed 42 selects $\tau^* = 0.24$, seed 7 selects $\tau^* = 0.36$, and seed 13 selects $\tau^* = 0.42$. Their test F1 at the chosen threshold is nevertheless consistent (mean 0.699, standard deviation 0.017), suggesting the F1 plateau is broad and the routers degrade gracefully when the threshold is mistuned.

Figure 4.4 shows the mean routed time on test as the threshold sweeps from 0 (every query routed to *Mix*) to 1 (every query routed to *Naive*). The always-*Naive* and always-*Mix* levels are shown as horizontal dashed lines and act as a sanity check. At $\tau = 0$ every router collapses to always-*Mix*, and at

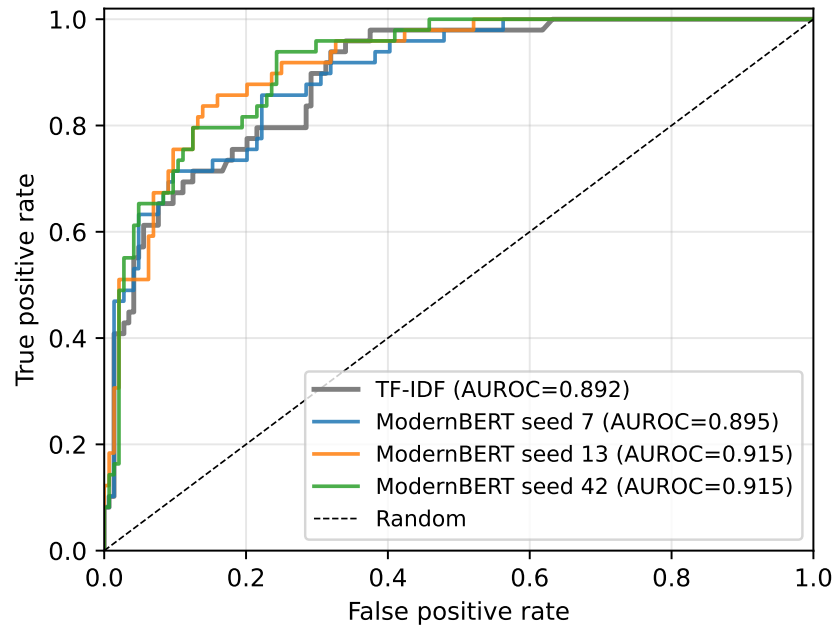


Figure 4.2: Receiver-operating-characteristic curves on the held-out test split.

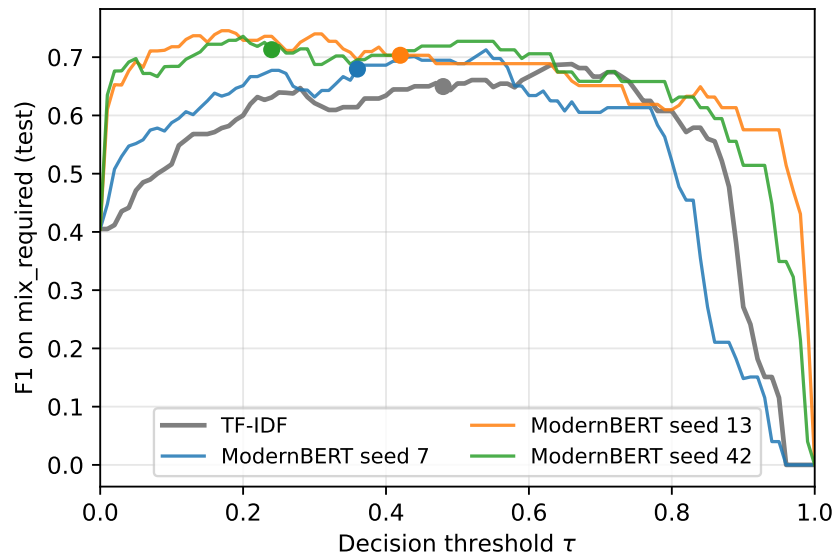


Figure 4.3: F1 on `mix_required` as a function of the decision threshold on the held-out test split. Dots mark the validation-selected threshold for each model: TF-IDF $\tau^* = 0.48$, ModernBERT seed 42 $\tau^* = 0.24$, seed 7 $\tau^* = 0.36$, seed 13 $\tau^* = 0.42$.

$\tau = 1$ every router collapses to always-*Naive*, which is what the figure shows.

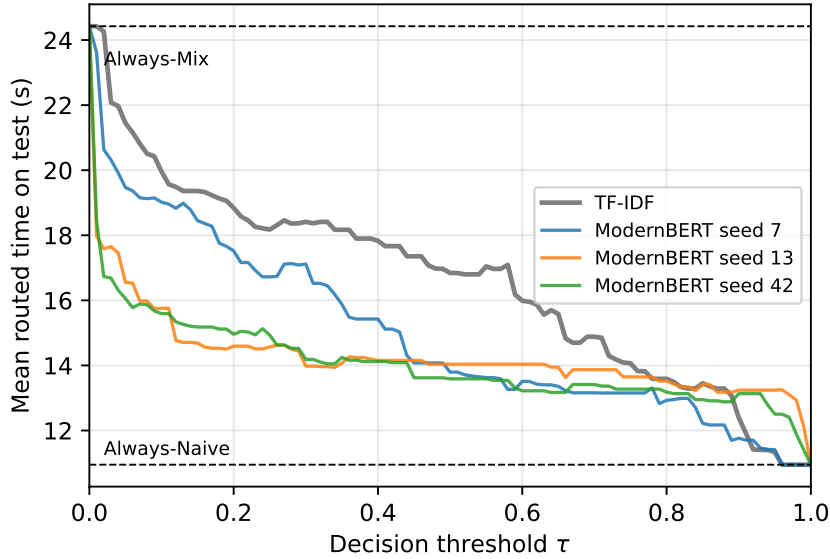


Figure 4.4: Mean routed execution time on the held-out test split as the decision threshold sweeps. The two horizontal dashed lines are the always-*Naive* and always-*Mix* reference levels.

Figure 4.5 combines the two preceding views into a routing-label-versus-routed-time frontier. Each router traces an operating curve in (mean routed time, $F1_{\text{mix}}$) space as τ sweeps. The static always-*Naive* and always-*Mix* policies appear as the two extreme operating points. The learned routers achieve higher routing-label $F1$ than either static policy at substantially lower mean routed time than always-*Mix*. The ModernBERT router lies slightly above and to the right of the **TF-IDF** baseline, reaching higher $F1_{\text{mix}}$ at a slightly higher mean routed time, with the type-only diagnostic baseline placed nearby for reference.

Table 4.3 reports a per-question-type breakdown on the test split. The 2WikiMultihopQA question types have very different routing-label distributions. The `comparison` slice is almost always `naive_enough`, with only 2 of 90 test rows judged *Mix*-beneficial. The `bridge_comparison` slice sits around one quarter. The `compositional` slice is the one in which the routing decision matters most, with 38 of 65 test rows judged *Mix*-beneficial. The `inference` slice contributes only 5 test rows, of which one is `mix_required`, so its per-class metrics are reported for completeness but should be read as a description of these particular test rows rather than as a stable estimate. On the `compositional` slice ModernBERT achieves

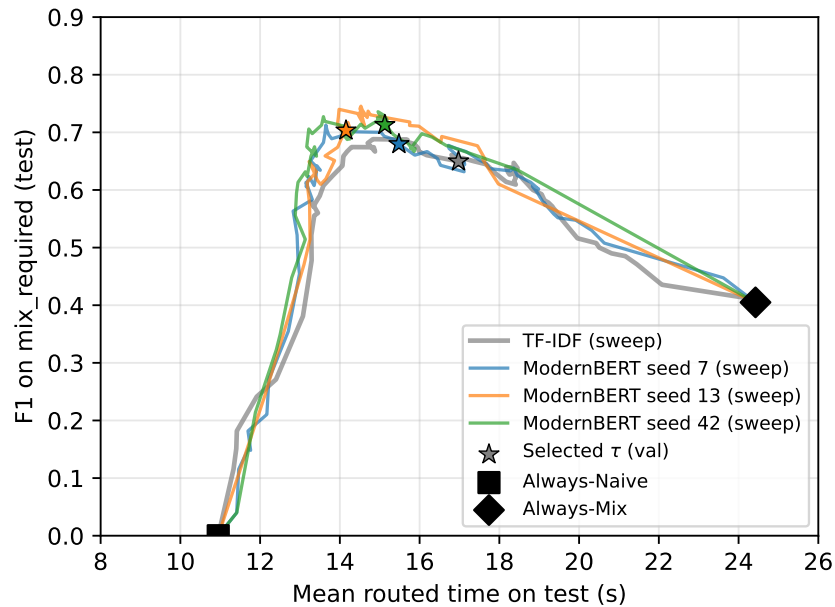


Figure 4.5: Routing-label F1 on the `mix_required` class versus mean routed execution time on the held-out test split. Each curve traces the operating points obtained by sweeping the decision threshold. Stars mark the validation-selected thresholds. Static *always-Naive* and *always-Mix* policies are shown as reference points.

higher point estimates than **TF-IDF** on **AUPRC** (mean 0.839 vs. 0.814) and F1 on `mix_required` (mean 0.799 vs. 0.753). On `bridge_comparison` the picture is more mixed. **TF-IDF** achieves a slightly higher **AUPRC** and F1 here (0.418 and 0.316) by routing more queries to *Mix* on this slice, while ModernBERT is more conservative, with lower recall (0.167 vs. 0.375) but higher precision (0.345 vs. 0.273) and higher overall slice accuracy (0.707 vs. 0.606).

Table 4.3: Per-question-type breakdown on the test split. P_{mix} , R_{mix} , and $F1_{\text{mix}}$ denote precision, recall, and F1 on the `mix_required` class. **AUPRC** values on the `comparison` and `inference` slices are computed on too few `mix_required` positives to be informative and should be read as a description of these particular test rows rather than as a stable estimate. ModernBERT values are the mean over three seeds.

Slice	n	n _{mix}	Model	AUPRC	P _{mix}	R _{mix}	F1 _{mix}	Acc.
comparison	90	2	TF-IDF	0.277	0.000	0.000	0.000	0.978
			ModernBERT	0.237	0.000	0.000	0.000	0.978
compositional	65	38	TF-IDF	0.814	0.636	0.921	0.753	0.646
			ModernBERT	0.839	0.741	0.868	0.799	0.744
bridge_comparison	33	8	TF-IDF	0.418	0.273	0.375	0.316	0.606
			ModernBERT	0.363	0.345	0.167	0.211	0.707
inference	5	1	TF-IDF	0.333	0.000	0.000	0.000	0.400
			ModernBERT	0.833	0.000	0.000	0.000	0.733

Figure 4.6 visualizes the **AUPRC** comparison on the two non-degenerate slices.

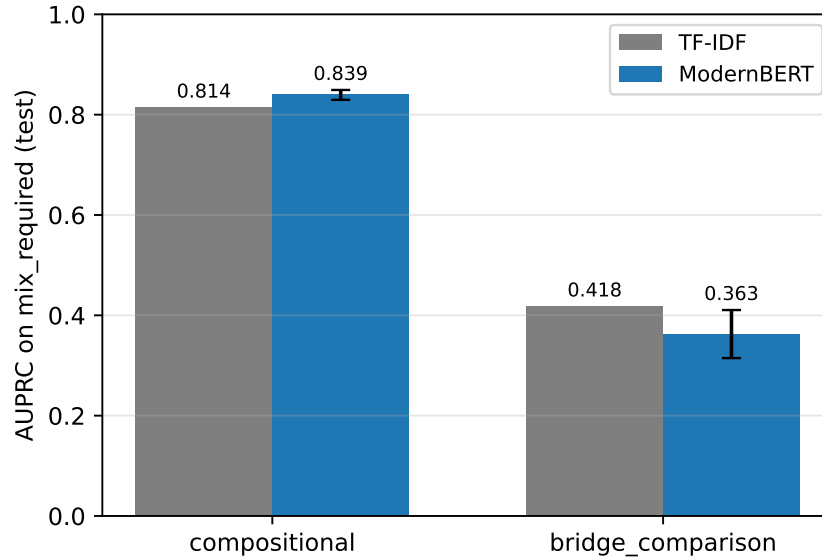


Figure 4.6: **AUPRC** on the `mix_required` class, broken down by 2WikiMultihopQA question type on the held-out test split. ModernBERT bars show the mean over three training seeds with error bars at one standard deviation. The `comparison` and `inference` slices have too few `mix_required` test rows to give an informative **AUPRC** and are omitted.

4.5 Discussion

The headline result is that the retrieval-mode distinction between *Naive* and *Mix* is partly predictable from the query text alone. Both routers outperform the always-*Naive* baseline on the minority class and reach operating points that no static policy at comparable cost can reach in Figure 4.5. The ModernBERT router achieves higher point estimates than the **TF-IDF** baseline on **AUPRC**, balanced routing-label accuracy, F1 on the minority class, and routing-label accuracy, with differences that are stable across three training seeds.

Routing errors: answer risk versus cost waste

The two routing error types have different practical meanings. A false negative routes a *Mix*-beneficial query to *Naive* and is therefore the error type most

directly associated with answer-quality risk. A false positive routes a *Naive*-sufficient query to *Mix* and is primarily a cost error, since the cheaper mode was already judged sufficient. This distinction explains why *always-Mix* has poor routing-label accuracy but is not necessarily poor in final answer quality. It also explains the difference between **TF-IDF** and ModernBERT in Table 4.2. **TF-IDF** has higher recall and a lower under-routing rate, and therefore fewer under-routed *Mix*-beneficial queries, while ModernBERT has higher precision and a lower over-routing rate, and therefore fewer unnecessary *Mix* calls.

Routing-label performance and routed cost should therefore be interpreted together. A router with high recall behaves conservatively and protects against missed *Mix*-beneficial queries, but may spend more. A router with high precision behaves selectively and saves more cost, but may accept more under-routing risk. The selected ModernBERT operating point is closer to the selective policy, while the **TF-IDF** operating point is closer to the conservative policy. Neither extreme is universally correct, and which one is preferable depends on the relative cost of an unnecessary *Mix* call versus an answer-quality regression in a given deployment.

Per-question-type behavior and question-type structure

The most informative slice is `compositional`. This is where the class distribution is balanced and where *Mix* is judged beneficial most often. On this slice ModernBERT achieves a mean **AUPRC** of 0.839 against **TF-IDF**'s 0.814 and lifts F1 on `mix_required` from 0.753 to 0.799, despite the small training set, which suggests that the encoder picks up on more than the lexical patterns a **TF-IDF** classifier can exploit. The `bridge_comparison` slice is harder for both routers. **TF-IDF** reaches a slightly higher **AUPRC** and F1 here (0.418 and 0.316) by routing more queries to *Mix* on this slice, while ModernBERT is more conservative on this slice and ends up with higher slice accuracy (0.707 vs. 0.606) and higher precision but lower minority recall. Neither outcome is satisfactory, and improving routing on this slice is an obvious target for further work.

The diagnostic question-type baseline confirms that part of the routing signal is explained by benchmark structure. Predicting *Mix* for `compositional` questions and *Naive* for the rest is already competitive on Table 4.1, because `compositional` questions dominate the *Mix*-beneficial class on this benchmark. This does not make the learned router unnecessary, because the question-type label is benchmark metadata and is not available in real user

deployments. The result instead shows that ModernBERT learns a deployable approximation of this structure from query text alone and achieves higher point estimates than the coarse metadata rule on F1, precision, and routing-label accuracy.

This affects how the results should be interpreted. The router is not learning a universal law of graph necessity. It is learning patterns in how questions are phrased that correlate with whether *Mix* is judged beneficial under this LightRAG setup and benchmark distribution. The type-only diagnostic baseline is therefore best read as an upper bound on what coarse benchmark structure alone can deliver. Anything ModernBERT adds beyond that bound is a credit to the encoder, but anything it does not exceed is more a description of the benchmark than of any deeper property of the queries.

Benchmark slices versus real deployments

The per-question-type breakdown and the type-only diagnostic baseline both rely on 2WikiMultihopQA’s question-type metadata. In a real deployment that metadata does not exist. Users send free-form queries, and the routing layer sees only the query text. The benchmark slices are useful here as a diagnostic device. They give a way to ask which subpopulations the router is learning to handle well and which it is not, and they help separate signal that comes from coarse structural cues (such as whether the question is a compositional multi-hop lookup) from signal that comes from finer phrasing. They are not a deployment recipe. The point of the learned routers is that they can approximate this structure from the query text alone, which is the only input available outside the benchmark.

This framing is also useful for interpreting the `compositional-versus-bridge_comparison` contrast. The fact that ModernBERT is strong on `compositional` and weaker on `bridge_comparison` tells us something about which phrasings the encoder has learned to recognize. It does not tell us that compositional queries are intrinsically easier in production, only that within this benchmark there are more of them and they share a more consistent surface form.

Routing-label accuracy versus final answer accuracy

This thesis does not report independently re-judged final answer accuracy for each routed policy. The offline simulation selects between precomputed *Naive* and *Mix* outputs, but the primary stored supervision is the routing

label. Routing-label accuracy should therefore not be read as final answer accuracy. In particular, *always-Mix* is label-incorrect on *Naive-sufficient* rows because it uses more retrieval than necessary, not because its answer is necessarily wrong. Direct policy answer accuracy would require retained per-mode correctness indicators for every query, or an additional judging pass over the answer selected by each policy.

The reported metrics are still useful for adaptive routing. Routing-label F1 measures whether the router selects the minimum judged sufficient mode. Mean routed time and estimated token usage measure the cost of doing so, and the under-routing and over-routing rates further separate answer-risk errors from cost-waste errors. Treated together, these metrics describe how often the router agrees with the judge and how expensive its agreement is.

Statistical caveats

There are limitations to the headline result. The held-out test set has 193 queries, which is enough to compute meaningful bootstrap intervals but not enough to resolve small differences between the routers. **TF-IDF**'s bootstrap **AUROC** interval ([0.836, 0.936]) contains the ModernBERT mean **AUROC** of 0.908, so the **AUROC** comparison cannot be called significant on this test set. The **AUPRC** and balanced-routing-label-accuracy gaps are larger but still inside the bootstrap intervals, which is what should be expected on a 193-row test set. The multi-seed standard deviation for ModernBERT is small on the aggregate routing-label metrics (≤ 0.017 on **AUPRC**, **AUROC**, balanced routing-label accuracy, F1, and routing-label accuracy), so the gains are at least stable across seeds. The per-class precision and recall vary more across seeds (standard deviations of 0.057 and 0.042 respectively), which is consistent with each seed selecting a slightly different threshold on a 193-row test set. A larger benchmark would be needed before stronger claims can be made.

Cost picture in time versus tokens

The cost picture is more pronounced in model tokens than in execution time. The two static baselines bracket a mean routed range of 11.0 to 24.4 seconds per query in execution time and roughly 3,500 to 17,900 tokens per query in model-token usage. The learned routers select operating points around 15 to 17 seconds (a 30–39 % reduction in time relative to *always-Mix*) and 7,100 to 8,500 tokens (a 52–60 % reduction in tokens relative to *always-Mix*), at non-trivial `mix_required` F1. The token saving is more dramatic than the time

saving because *Mix* adds one extra LLM call per query and a substantially larger assembled context [3, 12], whereas time is partly absorbed by network latency that is independent of token volume. The routing decision itself contributes essentially zero LLM tokens, since neither router calls a generative model. It contributes only milliseconds of compute time, which is negligible next to retrieval cost on either dimension.

Chapter 5

Conclusions and future work

This chapter summarizes the conclusions from the experiments (Section 5.1), discusses the main limitations and natural avenues for future work (Section 5.2), and reflects on the broader context of the project (Section 5.3).

5.1 Conclusions

This thesis studied whether the choice between LightRAG’s cheap *Naive* retrieval mode and its more expensive *Mix* retrieval mode can be predicted from the query text alone, on the 2WikiMultihopQA benchmark, using a lightweight encoder-only classifier rather than a large language-model controller. Routing labels were derived by executing both retrieval modes on each benchmark question and using an **LLM-as-a-Judge** [18] to compare each mode’s answer against the gold answer. The label `mix_required` is therefore operational. It means *Mix* was judged correct when *Naive* was not under the chosen LightRAG configuration and judge prompt, not that graph traversal alone caused the improvement.

On the held-out test split, the ModernBERT router achieves higher point estimates than a **TF-IDF** logistic-regression baseline on **AUPRC** (0.769 vs. 0.742), balanced routing-label accuracy (0.798 vs. 0.784), F1 on the minority `mix_required` class (0.699 vs. 0.650), and routing-label accuracy (0.846 vs. 0.788), while remaining within the bootstrap interval on **AUROC**. The multi-seed standard deviation of the ModernBERT router is small on the aggregate routing-label metrics (≤ 0.017 on **AUPRC**, **AUROC**, balanced routing-label accuracy, F1, and routing-label accuracy), which makes the differences stable rather than seed-specific. Both learned routers reach test-set operating points around 15 to 17 seconds of mean routed execution

time, compared with 11.0 seconds for *always-Naive* and 24.4 seconds for *always-Mix*, and achieve higher routing-label F1 than either static policy at substantially lower mean routed time than *always-Mix*. Measured in model tokens, the same routers route at roughly 7,100 to 8,500 mean tokens per query, compared with 3,458 for *always-Naive* and 17,857 for *always-Mix*, a routing saving of 52–60 % relative to the expensive baseline. The error decomposition tells a complementary story. ModernBERT trades a slightly higher under-routing rate (0.076 ± 0.009 vs. 0.057 for **TF-IDF**) for a substantially lower over-routing rate (0.078 ± 0.019 vs. 0.155 for **TF-IDF**), which is the source of its token saving.

The per-question-type breakdown localizes most of the routing signal in the `compositional` slice, where *Mix* was judged beneficial on 58 % of the test rows and where ModernBERT achieves a roughly two-and-a-half-point higher **AUPRC** than **TF-IDF**. The `comparison` slice is essentially decidable as *Naive* and both routers correctly avoid *Mix* on it. The `bridge_comparison` slice is the weakest part of the result and is the natural target for further work. The type-only diagnostic baseline shows that a substantial part of this signal is explained by 2WikiMultihopQA’s question-type metadata alone. ModernBERT achieves higher point estimates than that metadata rule on F1, precision, and routing-label accuracy while using only the query text, which is the input available in a real deployment.

Returning to the research question, the *Naive*-sufficient versus *Mix*-beneficial distinction is partly predictable from query text alone in this setting. ModernBERT achieves higher point estimates than **TF-IDF** and static baselines on several routing-label metrics while reducing routed token cost substantially relative to *always-Mix*. These results support learned query routing as a promising cost-control layer for Hybrid **RAG**, but the reported accuracy values are routing-label accuracies rather than independently measured final answer accuracies.

The learned router is not best understood as proving that graph retrieval is intrinsically required for particular question types. It predicts an operational label, namely whether LightRAG *Mix* was judged beneficial relative to *Naive* under a fixed benchmark, index, generator, and judge prompt. Within that scope, the result is positive. The router can often identify when the expensive path is unnecessary, and can therefore reduce routed cost without always defaulting to the most expensive retrieval mode.

5.2 Limitations and future work

5.2.1 Token-cost measurement scope

Model-token cost in this thesis is estimated using per-mode mean token usage from an instrumented subset of $N = 150$ queries spanning all four 2WikiMultihopQA question types. This is stronger than a very small pilot sample, but it is still not the same as per-query token logging across the full 1,286-row routing dataset. The reported token costs should therefore be interpreted as estimated routed token usage rather than exact measured token usage for every held-out query.

Future work should log token usage for every *Naive* and *Mix* execution in the full 1,286-row routing dataset. This would make it possible to compute per-query routed token cost directly, report uncertainty intervals for token cost, and study whether token usage varies systematically by question type or routing label.

5.2.2 Judge labels

Routing labels come from an **LLM-as-a-Judge** and are treated as judged supervision rather than absolute ground truth. Although the judge sees the gold answer and the same prompt is used consistently across the dataset [18], judge errors may still affect the labels. This is especially relevant for borderline answers, partially correct answers, and answers that use different surface forms from the gold answer.

The label names should also be interpreted operationally. A `mix_required` label means that *Mix* was judged correct when *Naive* was not, under the chosen LightRAG configuration and judge prompt. It does not prove that the graph component alone caused the improvement, because *Mix* also changes context assembly, retrieval breadth, and the evidence exposed to the generator.

Future work should validate a sample of judge decisions manually, compare multiple judge models, or use stricter answer-normalization rules where possible.

5.2.3 Final answer accuracy of routed policies

The offline policy evaluation reports routing-label performance and routed cost, but does not independently re-judge the final answer selected by each routed policy. This matters because false-positive *Mix* decisions are

counted as routing-label errors even though the *Mix* answer may still be correct. Conversely, false-negative *Mix* decisions are the errors most directly associated with answer-quality risk, because the router selects *Naive* on a query where *Mix* was judged necessary.

A future evaluation should retain per-mode correctness indicators for every query, or re-run the judge on the answer selected by each policy, so that judged final answer accuracy can be reported directly alongside routed cost. That would allow the main trade-off to be stated as answer correctness against token cost, rather than routing-label performance against token cost.

5.2.4 Generalizability

The experiments use one Hybrid **RAG** framework (LightRAG), two of its retrieval modes (*Naive* and *Mix*), and one public benchmark (2WikiMulti-hopQA). The held-out test set has 193 rows, which is enough to compute bootstrap intervals but not enough to resolve small differences between routers. The conclusions therefore apply most directly to the studied setting, and applying the same methodology to other Hybrid **RAG** frameworks (for example GraphRAG [2] or Hybrid retrieval modes in other systems) or to other multi-hop benchmarks (HotpotQA, MuSiQue [9, 10]) is a natural way to test how robust the routing signal is. Extending the routing labels to industrial corpora (for example Ericsson internal documentation) was considered but is blocked by the absence of per-query ground-truth answers in those corpora, which is what made the judge-labeling protocol possible on 2WikiMultihopQA in the first place.

The strong question-type diagnostic baseline also shows that part of the routing signal may be benchmark-specific. Results should therefore not be interpreted as proving that the same router would transfer unchanged to arbitrary industrial queries without additional validation.

5.2.5 Routing beyond the query

The router used in this thesis is query-only by construction. A router that also has access to some lightweight signal from the corpus or from a cheap first retrieval pass could in principle make a better decision, at the cost of a slightly more expensive routing layer. A natural follow-up is to compare the query-only router against a hybrid scheme that first runs the cheap *Naive* mode, inspects whether the retrieved chunks contain the entities referenced in the query, and then escalates to *Mix* only when this signal looks weak, similar to

how Adaptive-RAG conditions on cheap intermediate signals at the language-model level [4]. This would still keep the routing layer cheap (one round of vector retrieval rather than an expensive graph traversal) but could materially improve the harder slices, in particular `bridge_comparison`.

5.2.6 Calibration and abstention

The router currently outputs a single thresholded decision. Routing under cost constraints is a natural setting for selective prediction [24] and calibrated probabilities [25], where the router could abstain on low-confidence queries (for example by escalating to a more expensive but more reliable controller, or by running both retrieval modes and combining the answers). The calibration of the ModernBERT router’s softmax outputs was not investigated in this work and is a natural piece of follow-up.

5.3 Reflections

This thesis is centered on making a Hybrid **RAG** system more resource-efficient. The most direct connection is to the **United Nations (UN) Sustainable Development Goal (SDG)** number 12 (responsible consumption and production). Graph-enhanced retrieval is significantly more compute-intensive than vector retrieval [3, 12], and a router that avoids applying it to queries that do not need it directly reduces the energy and compute footprint of the system at inference time. There is also a softer connection to **SDG** number 9 (industry, innovation, and infrastructure), in that adaptive routing is a building block for industrial **RAG** deployments such as the one motivating this work at Ericsson, where serving company-internal documentation at scale is a real operational concern.

The work has some ethical aspects worth flagging. The routing labels are produced by an **LLM-as-a-Judge** that has access to the gold answer. This is a methodologically reasonable but imperfect protocol, and the thesis is explicit about not treating these labels as ground truth. Building a system that decides on a per-query basis which retrieval path to use could in principle interact with downstream notions of answer quality in ways that are hard to anticipate. If the router systematically routes some kind of question to the cheaper path, that kind of question could end up consistently worse-served, even when the average behavior of the system improves. The per-question-type breakdown in Section 4.4 is one way to surface this kind of differential behavior, and

any future work that scales this approach to user-facing settings should keep similar per-segment analyses in view.

References

- [1] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.10997>
- [2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, “From local to global: A graph rag approach to query-focused summarization,” 2025. [Online]. Available: <https://arxiv.org/abs/2404.16130>
- [3] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “Lightrag: Simple and fast retrieval-augmented generation,” 2025. [Online]. Available: <https://arxiv.org/abs/2410.05779>
- [4] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, “Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024. doi: 10.18653/v1/2024.naacl-long.389 pp. 7036–7050. [Online]. Available: <https://aclanthology.org/2024.naacl-long.389/>
- [5] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay, “Routerbench: A benchmark for multi-llm routing system,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.12031>
- [6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-rag: Learning to retrieve, generate, and critique through self-reflection,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.11511>

- [7] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, “Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps,” in *Proceedings of the 28th International Conference on Computational Linguistics*, D. Scott, N. Bel, and C. Zong, Eds. Barcelona, Spain (Online): International Committee on Computational Linguistics, Dec. 2020. doi: 10.18653/v1/2020.coling-main.580 pp. 6609–6625. [Online]. Available: <https://aclanthology.org/2020.coling-main.580/>
- [8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [9] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018. doi: 10.18653/v1/D18-1259 pp. 2369–2380. [Online]. Available: <https://aclanthology.org/D18-1259/>
- [10] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “MuSiQue: Multihop questions via single-hop question composition,” *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022. doi: 10.1162/tacl_a_00475. [Online]. Available: <https://aclanthology.org/2022.tacl-1.31/>
- [11] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, B. Webber, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020. doi: 10.18653/v1/2020.emnlp-main.550 pp. 6769–6781. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550/>
- [12] L. Stamatescu, “GraphRAG costs explained: What you need to know,” Microsoft Foundry Blog, Aug. 2024, accessed 2026-05-04. [Online]. Available: <https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/graphrag-costs-explained-what-you-need-to-know/4207978>

- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008. [Online]. Available: <https://papers.nips.cc/paper/7181-attention-is-all-you-need>
- [14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019. doi: 10.18653/v1/N19-1423 pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>
- [15] A. Benayas, M. A. Sicilia, and M. Mora-Cantallos, “A comparative analysis of encoder only and decoder only models in intent classification and sentiment analysis: navigating the trade-offs in model size and performance,” *Language Resources and Evaluation*, vol. 59, no. 3, pp. 2007–2030, 2024. doi: 10.1007/s10579-024-09796-y. [Online]. Available: <https://doi.org/10.1007/s10579-024-09796-y>
- [16] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini, A. Gallagher, R. Biswas, F. Ladhak, T. Aarsen, N. Cooper, G. Adams, J. Howard, and I. Poli, “Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.13663>
- [17] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988. doi: 10.1016/0306-4573(88)90021-0
- [18] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging llm-as-a-judge with mt-bench and chatbot arena,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [19] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [20] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers

- on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015. doi: 10.1371/journal.pone.0118432. [Online]. Available: <https://doi.org/10.1371/journal.pone.0118432>
- [21] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [22] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” *Mathematical Programming*, vol. 45, no. 1, pp. 503–528, 1989. doi: 10.1007/BF01589116
- [23] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. doi: 10.1214/aos/1176344552
- [24] Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1705.08500>
- [25] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 1321–1330. [Online]. Available: <https://proceedings.mlr.press/v70/guo17a.html>

Appendix A

Source code

The implementation code, the data-preparation pipeline, the training scripts, and the evaluation scripts used in this thesis are publicly available at https://github.com/Dantecool02/Thesis_ericsson.

Appendix B

Experimental configuration

This appendix lists the configuration used to produce the routing labels, train the routers, and compute the reported routed-cost estimates. Table [B.1](#) summarizes the LightRAG, data, and judge settings. Table [B.2](#) summarizes the router training settings. The exact judge prompt is reproduced in Section [B.1](#).

B.1 Judge prompt

The judge receives the question, the gold answer, the *Naive* answer, and the *Mix* answer in a JSON payload, and is asked to return exactly one of the three class labels `naive_enough`, `mix_required`, or `none_enough`. The system prompt is reproduced verbatim below.

```
You are a strict evaluator for a retrieval-routing
dataset.
You must output exactly one class label and nothing else
.
Allowed labels:
- naive_enough
- mix_required
- none_enough
```

The label is assigned as follows. `naive_enough` means the *Naive* answer matches the gold answer. `mix_required` means the *Naive* answer does not match the gold answer but the *Mix* answer does. `none_enough` means neither answer matches the gold answer and the row is excluded from the binary routing dataset.

Table B.1: LightRAG, data, and judge configuration used in this thesis. The same generator model and embedding model are used during indexing and at query time. All Gemini calls use a temperature of 0.0.

Item	Value
Generator model	gemini-3-flash-preview (Gemini API)
Judge model	gemini-3-flash-preview (Gemini API)
Embedding model	models/gemini-embedding-001 (768-dim)
LightRAG version	1.4.10 (vendored fork in external/- LightRAG)
LightRAG chunk token size	1 200 tokens
LightRAG chunk overlap	100 tokens
<i>Naive</i> retrieval parameters	top_k=3, chunk_top_k=3
<i>Mix</i> retrieval parameters	mode="mix", chunk_top_k=3; entity- relation graph built during indexing
LightRAG tokenizer	gpt-4o-mini via tiktoken
Generator temperature	0.0
Judge temperature	0.0
Token logging method	Gemini API usage_metadata per LLM call, plus tiktoken estimates for embedding calls when the API does not report tokens
Token measurement sub- set	$N = 150$ queries spanning all four 2WikiMultihopQA question types
Judge prompt location	Section B.1

Table B.2: Router training configuration used in this thesis.

Item	Value
TF-IDF features	Unigrams and bigrams, lowercase, English stop-words removed, min document frequency 2, vocabulary cap 20 000
TF-IDF classifier	Logistic regression, L_2 regularization with $C = 1.0$, balanced class weights, scikit-learn L-BFGS solver, max 1 000 iterations
ModernBERT checkpoint	answerdotai/ModernBERT-base
ModernBERT max sequence length	128 tokens
ModernBERT learning rate	2×10^{-5}
ModernBERT weight decay	0.01
ModernBERT batch size	Per-device 8, gradient accumulation 2, effective batch 16
ModernBERT optimizer	AdamW with linear warm-up over 10 % of steps and linear decay; gradient clip 1.0
ModernBERT max epochs	8, early stopping on validation $F1_{\text{mix}}$ with patience 3, minimum 3 epochs
ModernBERT seeds	7, 13, 42
Split sizes	900 train / 193 validation / 193 test
Split strategy	Joint stratification by question type and routing label
Hardware	Single CPU; no GPU used for ModernBERT training or inference

